Course Project ML-1:
Learning Plaintext–Ciphertext Relations using Machine Learning

Project Overview

Classical cryptanalysis techniques such as differential and linear cryptanalysis exploit structured relations between plaintexts and ciphertexts in order to distinguish a cipher from a random function. Recent advances in machine learning (ML) have shown that neural models can *automatically learn* such relations without being explicitly provided with cryptanalytic rules. These techniques form the foundation of modern *neural cryptanalysis*. In this project, students will apply ML techniques to learn and exploit plaintext–ciphertext relations in block ciphers and construct relation-based distinguishers.

1 Problem Statement

Let F be an unknown function that is either:

(a) A reduced-round block cipher E_k, or

(b) A uniformly random permutation π.

You are given access to pairs of inputs with a fixed XOR difference

$$(P, P \oplus \Delta P),$$

and their corresponding outputs

$$(C, C') = (F(P), F(P \oplus \Delta P)).$$

Your task is to design an ML-based distinguisher that determines whether F is a real cipher or a random permutation by learning exploitable relations between plaintexts and ciphertexts.

2 Cryptographic Setup

Students may choose **one** of the following primitives, or any other similar ciphers:

- A lightweight cipher (e.g., SIMON, SPECK, PRESENT)
- A stream cipher
- A masked or tweakable construction (e.g., Even–Mansour)

The same setup must be used for both the cipher and the random permutation baseline.

3 Dataset Generation

You are required to generate labeled datasets as follows:

- Sample random plaintexts P
- Compute paired plaintexts $P' = P \oplus \Delta P$
- Obtain ciphertext pairs (C, C')

Each sample must be labeled as:

$$y = \begin{cases} 1 & \text{Cipher-generated} \\ 0 & \text{Random-generated} \end{cases}$$

4 Input Representations

The choice of input representation plays a critical role in ML-based cryptanalysis. Students are encouraged to experiment with multiple representations and analyze their impact on distinguisher performance.

Possible input representations include, but are not limited to:

1. **Raw Ciphertext Pairs**
$$(C, C')$$
2. **Ciphertext Difference**
$$\Delta C = C \oplus C'$$
3. **Concatenated Representation**
$$(C \parallel C')$$
4. **Bit-Sliced Representation**

   Represent each ciphertext bit as a separate feature channel, enabling convolutional models to learn local and global bit dependencies.
5. **Word-Level Representation**

   For word-oriented ciphers, represent ciphertexts as vectors of words instead of individual bits.
6. **Intermediate Differences (White-Box Setting)**

   For experimental analysis, include intermediate round differences:
$$\Delta R_i = R_i(P) \oplus R_i(P \oplus \Delta P)$$
   to study differential propagation across rounds.
7. **Masked or Noisy Representations**

   Introduce random masks or noise to:

- Test robustness of learned relations
- Study resistance to countermeasures

8. **Joint Plaintext–Ciphertext Representation**

$$(P, C, P \oplus \Delta P, C')$$

9. **Statistical Feature Vectors**

   Construct features such as:

   - Hamming weight of ΔC
   - Bit correlation frequencies
   - Empirical probability distributions

10. **Sequential / Round-Wise Representation**

    For iterative ciphers, represent differences as a sequence:

$$(\Delta R_1, \Delta R_2, \ldots, \Delta R_r)$$

    suitable for recurrent or attention-based models.

Students must justify their chosen representation(s) and discuss how they influence model effectiveness and interpretability.

5 Machine Learning Models

You must implement and compare **at least three** of the following models:

1. Multi-Layer Perceptron (MLP)
2. Convolutional Neural Network (CNN)
3. Siamese or twin-network architectures
4. **Mutual Information Neural Estimator (MINE)**

You must identify the *maximum number of rounds* for which a relation-based distinguisher remains effective.

Optional Extensions (Bonus)

- Automatically search for effective input differences
- Compare ML-based and classical differential distinguishers
- Apply transfer learning across round numbers or ciphers
- Extend the approach toward limited key-recovery experiments

Deliverables

- Dataset generation code
- ML model implementations and training scripts
- Experimental results and plots
- A written report (6–10 pages)